

SISTEMA DE GESTÃO COMERCIAL

Assistência Técnica — TechFix

Entrega 1 — Modelagem e Arquitetura

Disciplina: Construção e Configuração de Software

Campo	Informação
Sistema	SGC — Sistema de Gestão Comercial
Contexto	Assistência Técnica (TechFix)
Entrega	1 — Modelagem e Arquitetura
Data de entrega	02/04/2025
Tecnologias base	Java 21 · Spring Boot 3 · MySQL · JWT

1. Descrição do Sistema

O **SGC TechFix** é um sistema de gestão comercial desenvolvido para atender às necessidades operacionais de uma assistência técnica de pequeno porte. O contexto real contempla o atendimento a clientes que entregam equipamentos eletrônicos (computadores, notebooks, celulares e periféricos) para diagnóstico, reparo e manutenção.

O sistema busca resolver problemas recorrentes nesse segmento: controle manual de ordens de serviço em papel, perda de histórico de atendimentos, dificuldade de rastreamento de peças e estoque, e falta de visibilidade financeira. A solução proposta digitaliza e integra todos esses processos em uma plataforma única, com interface web e API REST.

Objetivos do sistema

- Gerenciar o cadastro de clientes e seus equipamentos
- Controlar ordens de serviço (OS) do diagnóstico até a entrega
- Gerenciar o estoque de peças e insumos
- Registrar vendas de produtos e serviços
- Gerar relatórios gerenciais de vendas e atendimentos por período
- Controlar acesso por perfil (ADMIN e FUNCIONÁRIO)

2. Requisitos Funcionais

ID	Nome	Descrição
RF01	Autenticação	O sistema deve permitir login com username e senha, retornando um token JWT com t
RF02	Gestão de Usuários	O sistema deve permitir cadastro, edição e inativação de usuários com perfis ADMIN ou
RF03	Gestão de Clientes	O sistema deve permitir cadastro, consulta, edição e exclusão de clientes. CPF deve se
RF04	Gestão de Equipamentos	O sistema deve permitir vincular equipamentos (tipo, marca, modelo, número de série)
RF05	Gestão de Produtos/Peças	O sistema deve permitir cadastrar produtos com nome, descrição, preço e quantidade e
RF06	Registro de Venda	O sistema deve permitir registrar vendas com cliente, lista de itens, data e valor total ca
RF07	Relatório por Período	O sistema deve gerar relatório de vendas filtrando por período (data início e fim).
RF08	Relatório por Cliente	O sistema deve listar todas as vendas associadas a um cliente específico.
RF09	Relatório Gráfico Anual	O sistema deve exibir representação gráfica das vendas agrupadas por mês no ano sel
RF10	Proteção de Rotas	Todos os endpoints, exceto /auth/login, devem exigir token JWT válido no cabeçalho Au

3. Requisitos Não Funcionais

ID	Categoria	Descrição
RNF01	Segurança	Senhas armazenadas com hash BCrypt. Autenticação via JWT com expiração configurável
RNF02	Desempenho	Consultas de listagem devem responder em até 2 segundos para bases com até 10.000 re
RNF03	Disponibilidade	O sistema deve ter disponibilidade mínima de 99% em ambiente de produção local.

ID	Categoria	Descrição
RNF04	Manutenibilidade	Código organizado em camadas com responsabilidades bem definidas. Baixo acoplamento.
RNF05	Portabilidade	O backend deve executar em qualquer sistema operacional com JDK 21+. Banco de dados suportado.
RNF06	Usabilidade	A interface deve ser responsiva e permitir conclusão das tarefas principais em no máximo 3 cliques.
RNF07	Interoperabilidade	Todos os endpoints REST devem retornar JSON com códigos HTTP semânticos (200, 201, 400, 401, 403, 404, 500).
RNF08	Rastreabilidade	Toda venda deve registrar o usuário responsável (funcionário) pelo registro.

4. Arquitetura Proposta

O sistema adota a **Arquitetura em Camadas** (Layered Architecture), padrão amplamente utilizado em aplicações corporativas baseadas em Spring Boot. Essa abordagem promove separação de responsabilidades, facilita testes unitários e reduz o acoplamento entre componentes.

Camada	Responsabilidade	Tecnologia/Classes
Apresentação (Frontend)	Interface gráfica com o usuário. Consome a API REST.	Web API REST (Spring MVC), Requisição HTTP.
Controller	Recebe as requisições HTTP, valida entrada, chama a camada de serviço.	ProdutoController, VendaController
Serviço (Service)	Contém as regras de negócio e orquestra chamadas para a camada de persistência.	ProdutoService, VendaService, AuthService
Domínio (Domain)	Define as entidades do negócio e os repositórios.	Cliente, Produto, ItemVenda, Usuario
Persistência (Repository)	Acesso ao banco de dados usando Spring Data JPA.	ProdutoRepository, etc.
Banco de Dados	Armazenamento persistente dos dados.	MySQL 8+

Fluxo de uma requisição

Cliente HTTP → **Controller** (valida token JWT, recebe DTO) → **Service** (aplica regras de negócio) → **Repository** (persiste/consulta dados) → **Banco de Dados** → resposta JSON ao cliente.

Segurança — JWT

- Endpoint público: POST /auth/login — retorna o token JWT
- Todos os demais endpoints exigem o header: Authorization: Bearer <token>
- O filtro JwtAuthenticationFilter intercepta todas as requisições e valida o token
- Controle de acesso por perfil via Spring Security: ADMIN tem acesso total; FUNCIONARIO tem acesso restrito

5. Padrões de Projeto

5.1 Repository Pattern (principal)

O padrão **Repository** é utilizado para abstrair o acesso ao banco de dados. Interfaces como `ClienteRepository` estendem `JpaRepository` do Spring Data, isolando a camada de serviço dos detalhes de persistência. Isso facilita a troca futura da fonte de dados e torna os serviços testáveis com mocks.

- **Onde:** pacote `domain/repository/`
- **Benefício:** desacoplamento entre lógica de negócio e mecanismo de persistência

5.2 DTO Pattern (complementar)

O padrão **DTO** (Data Transfer Object) é aplicado para separar os dados trafegados na API das entidades de domínio. Classes como `ClienteDTO` e `VendaDTO` controlam exatamente quais campos são expostos ou recebidos nas requisições, evitando vazamento de dados sensíveis (ex: senha) e desacoplando o contrato da API do modelo interno.

- **Onde:** pacote `dto/`
- **Benefício:** segurança, versionamento da API sem impacto no domínio

5.3 Singleton (via Spring IoC)

O Spring Boot gerencia todos os beans (`@Service`, `@Repository`, `@Component`) como Singletons por padrão, garantindo que apenas uma instância de cada componente exista no contexto da aplicação.

- **Onde:** todos os componentes gerenciados pelo Spring
- **Benefício:** economia de memória e consistência de estado

6. Diagrama de Domínio (Descrição Textual)

O diagrama de domínio representa as entidades centrais do negócio e seus relacionamentos. Abaixo a descrição de cada entidade e suas relações:

Entidade	Atributos principais	Relacionamentos
Usuario	id, username, senha, perfil (ADMIN/FUNCIONARIO)	1 Usuario registra N Vendas
Cliente	id, nome, cpf, email, telefone, endereco	1 Cliente possui N Equipamentos 1 Cliente realiza N Vendas
Equipamento	id, tipo, marca, modelo, numSerie	N Equipamentos pertencem a 1 Cliente
Produto	id, nome, descricao, preco, qtdEstoque	N Produtos aparecem em N Vendas (via ItemVenda)
Venda	id, data, valorTotal, cliente, usuario	1 Venda possui N ItensVenda 1 Venda pertence a 1 Cliente 1 Venda registrada por 1 Usuario
ItemVenda	id, quantidade, precoUnitario	N ItensVenda referenciam 1 Produto N ItensVenda pertencem a 1 Venda

Nota: O diagrama UML visual (formato .png ou .drawio) deve ser anexado ao repositório GitHub no diretório docs/diagramas/.

7. Diagrama de Classes (Descrição Textual)

As principais classes Java do sistema e seus membros relevantes:

Classe / Interface	Pacote	Membros principais
Usuario	domain.model	- id: Long - username: String - senha: String - perfil: PerfilUsuario (enum)
Cliente	domain.model	- id: Long - nome, cpf, email - telefone, endereco: String - vendas: List<Venda>
Produto	domain.model	- id: Long - nome, descricao: String - preco: BigDecimal - qtdEstoque: Integer
Venda	domain.model	- id: Long - data: LocalDateTime - valorTotal: BigDecimal - cliente: Cliente - usuario: Usuario - itens: List<ItemVenda>
ItemVenda	domain.model	- id: Long - produto: Produto - quantidade: Integer - precoUnitario: BigDecimal

Classe / Interface	Pacote	Membros principais
ClienteService	service	+ listar(): List<ClienteDTO> + buscarPorId(id): ClienteDTO + salvar(dto): ClienteDTO + atualizar(id, dto): ClienteDTO + deletar(id): void
VendaService	service	+ registrar(dto): VendaDTO + buscarPorId(id): VendaDTO + listarPorPeriodo(ini, fim): List + listarPorCliente(id): List
ClienteRepository	domain.repository	+ findByCpf(cpf): Optional<Cliente> + existsByVendas(id): boolean
JwtService	config	+ gerarToken(user): String + validarToken(token): boolean + extrairUsername(token): String
GlobalExceptionHandler	exception	+ handleBusiness(ex): ResponseEntity + handleNotFound(ex): ResponseEntity

8. Diagrama Lógico do Banco de Dados (Descrição Textual)

As tabelas, colunas, chaves primárias (PK) e estrangeiras (FK) do banco de dados relacional:

Tabela	Coluna	Tipo	Restrições
usuarios	id	BIGINT	PK, AUTO_INCREMENT
	username	VARCHAR(100)	NOT NULL, UNIQUE
	senha	VARCHAR(255)	NOT NULL
	perfil	ENUM('ADMIN','FUNCIONARIO')	NOT NULL
clientes	id	BIGINT	PK, AUTO_INCREMENT
	nome	VARCHAR(150)	NOT NULL
	cpf	VARCHAR(14)	NOT NULL, UNIQUE
	email	VARCHAR(150)	NOT NULL
	telefone	VARCHAR(20)	
	endereco	VARCHAR(255)	
produtos	id	BIGINT	PK, AUTO_INCREMENT
	nome	VARCHAR(150)	NOT NULL
	descricao	TEXT	
	preco	DECIMAL(10,2)	NOT NULL, CHECK >= 0
	qtd_estoque	INT	NOT NULL, DEFAULT 0
vendas	id	BIGINT	PK, AUTO_INCREMENT
	data	DATETIME	NOT NULL
	valor_total	DECIMAL(10,2)	NOT NULL
	cliente_id	BIGINT	FK → clientes(id)
	usuario_id	BIGINT	FK → usuarios(id)
itens_venda	id	BIGINT	PK, AUTO_INCREMENT
	venda_id	BIGINT	FK → vendas(id)
	produto_id	BIGINT	FK → produtos(id)
	quantidade	INT	NOT NULL, CHECK > 0
	preco_unitario	DECIMAL(10,2)	NOT NULL

Nota: O diagrama visual (formato .png ou gerado pelo MySQL Workbench) deve ser anexado no repositório em docs/diagramas/diagrama_banco.png.

9. Repositório GitHub

O repositório deve ser criado com a seguinte estrutura inicial:

github.com/<seu-usuario>/sgc-techfix

Diretório / Arquivo	Conteúdo
README.md	Descrição do projeto, tecnologias, instruções de execução
backend/	Código-fonte Spring Boot (estrutura conforme especificação)
docs/diagramas/	Diagrama de Classes, Domínio e Banco (.png ou .drawio)
docs/SGC_Entrega1.pdf	Este documento
sql/script_criacao.sql	Script SQL de criação do banco
.gitignore	Ignorar target/, .env, *.class, *.jar

Boas práticas de commits

- feat: adiciona entidade Cliente
- docs: adiciona diagrama de classes
- chore: configura projeto Spring Boot inicial
- sql: adiciona script de criação do banco

Apêndice — Checklist da Entrega 1

Item	Critério	Status
Descrição do sistema	Contexto real descrito (TechFix)	✓ Incluso
Requisitos funcionais	10 RFs definidos e detalhados	✓ Incluso
Requisitos não funcionais	5 RNFs com categorias (segurança, desempenho, etc.)	✓ Incluso
Arquitetura em camadas	6 camadas descritas com tecnologias	✓ Incluso
Design Patterns	Repository + DTO + Singleton justificados	✓ Incluso
Diagrama de Domínio	Entidades e relacionamentos descritos	✓ Pendente diagrama visual
Diagrama de Classes	Classes, pacotes e membros descritos	✓ Pendente diagrama visual
Diagrama lógico do banco	Tabelas, tipos e chaves descritos	✓ Pendente diagrama visual
Script SQL	Arquivo script_criacao.sql	✓ Arquivo separado
Repositório GitHub	Estrutura e commits iniciais	Criar e commitar